MACHINE LEARNING
LAB - 1

Question 1:

Create a NumPy array containing the integers from 1 to 20.

Perform the following operations:

1. Display the array.
2. Find the shape of the array.
3. Find the maximum and minimum values.
4. Calculate the mean and standard deviation.
5. Reshape the array into a 4 × 5 matrix.
6. Display the second row of the matrix.
7. Display the third column of the matrix.

Code:

```
import numpy as np

a = np.arange(1,21)

print("Array:")

print(a)

print("Shape:")

print(a.shape)

print("Maximum value:")

print(a.max())

print("Minimum value:")

print(a.min())

print("Mean:")

print(a.mean())

print("Standard Deviation:")

print(a.std())

b = a.reshape(4,5)

print("4 x 5 Matrix:")

print(b)

print("Second Row:")
```

```
print(b[1])

print("Third Column:")

print(b[:,2])
```

Output:

```
Array:
[ 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20]
Shape:
(20,)
Maximum value:
20
Minimum value:
1
Mean:
10.5
Standard Deviation:
5.766281297335398
4 x 5 Matrix:
[[ 1  2  3  4  5]
 [ 6  7  8  9 10]
 [11 12 13 14 15]
 [16 17 18 19 20]]
Second Row:
[ 6  7  8  9 10]
Third Column:
[ 3  8 13 18]
```

Question 2:

Create two NumPy arrays:

A = [10, 20, 30, 40, 50]

B = [5, 10, 15, 20, 25]

Perform:

1. Addition of A and B.
2. Subtraction of B from A.
3. Element-wise multiplication.
4. Element-wise division.
5. Find the sum of all elements in A

Code:

```
import numpy as np

A = np.array([10, 20, 30, 40, 50])

B = np.array([5, 10, 15, 20, 25])

print("Addition:")

print(A + B)

print("Subtraction:")

print(A - B)

print("Multiplication:")
```

```
print(A * B)

print("Division:")

print(A / B)

print("Sum of elements in A:")

print(A.sum())
```

Output:

```
Addition:
[15 30 45 60 75]
Subtraction:
[ 5 10 15 20 25]
Multiplication:
[  50  200  450  800 1250]
Division:
[2. 2. 2. 2. 2.]
Sum of elements in A:
150
```

Question 3:

Create a DataFrame using the following student data:

Roll No Name Marks

101 Ravi 85

102 Priya 92

103 Kiran 78

104 Anu 88

105 Sita 95

Perform the following operations:

1. Display the DataFrame.
2. Display the first three records.
3. Find the average marks.
4. Find the student with the highest marks.
5. Find the student with the lowest marks.
6. Add a new column called "Grade" using:
   A for Marks ≥ 90
   B for Marks between 80 and 89
   C for Marks < 80

Code:

```
import pandas as pd
```

```
data = {
    "Roll No": [101, 102, 103, 104, 105],
    "Name": ["Ravi", "Priya", "Kiran", "Anu", "Sita"],
    "Marks": [85, 92, 78, 88, 95]
}
df = pd.DataFrame(data)
print("DataFrame")
print(df)
print("\nFirst Three Records")
print(df.head(3))
print("\nAverage Marks")
print(df["Marks"].mean())
print("\nStudent with Highest Marks")
print(df[df["Marks"] == df["Marks"].max()])
print("\nStudent with Lowest Marks")
print(df[df["Marks"] == df["Marks"].min()])
grade = []
for m in df["Marks"]:
    if m >= 90:
        grade.append("A")
    elif m >= 80:
        grade.append("B")
    else:
        grade.append("C")
df["Grade"] = grade
print("\nDataFrame with Grade")
print(df)
```

Output:

```
DataFrame
   Roll No   Name  Marks
0      101   Ravi     85
1      102  Priya     92
2      103  Kiran     78
3      104    Anu     88
4      105   Sita     95

First Three Records
   Roll No   Name  Marks
0      101   Ravi     85
1      102  Priya     92
2      103  Kiran     78

Average Marks
87.6

Student with Highest Marks
   Roll No  Name  Marks
4      105  Sita     95

Student with Lowest Marks
   Roll No   Name  Marks
2      103  Kiran     78

DataFrame with Grade
   Roll No   Name  Marks Grade
0      101   Ravi     85     B
1      102  Priya     92     A
2      103  Kiran     78     C
3      104    Anu     88     B
4      105   Sita     95     A
```

Question 4:

Create a DataFrame containing the following information:

Product Price Quantity

Laptop 50000 10

Mouse 500 50

Keyboard 1200 30

Monitor 10000 15

Perform:

1. Calculate the total value of each product using:

Total Value = Price × Quantity

2. Add the Total Value column.

3. Find the product with the highest total value.

4. Calculate the overall inventory value.

Code:

```
import pandas as pd

data = [

   ["Laptop", 50000, 10],

   ["Mouse", 500, 50],
```

```
    ["Keyboard", 1200, 30],
    ["Monitor", 10000, 15]
]
df = pd.DataFrame(data, columns=["Product", "Price", "Quantity"])
df["Total Value"] = df["Price"] * df["Quantity"]
print("Product Data:")
print(df)
print("\nProduct with Highest Total Value:")
print(df[df["Total Value"] == df["Total Value"].max()])
print("\nOverall Inventory Value:")
print(df["Total Value"].sum())
```

Output:

```
Product Data:
    Product  Price  Quantity  Total Value
0    Laptop  50000        10       500000
1     Mouse    500        50        25000
2  Keyboard   1200        30        36000
3   Monitor  10000        15       150000

Product with Highest Total Value:
  Product  Price  Quantity  Total Value
0  Laptop  50000        10       500000

Overall Inventory Value:
711000
```

Question 5:

Download the Iris dataset from Scikit-learn and perform the following:

1. Load the dataset into a Pandas DataFrame.
2. Display the first five records.
3. Find the number of rows and columns.
4. Display column names.
5. Check for missing values.
6. Generate summary statistics using describe().
7. Find the number of samples belonging to each class.

Code:

```
from sklearn.datasets import load_iris

import pandas as pd
```

```
iris = load_iris()

df = pd.DataFrame(iris.data, columns=iris.feature_names)

df["Target"] = iris.target

print("First Five Records:")

print(df.head())

print("\nNumber of Rows and Columns:")

print(df.shape)

print("\nColumn Names:")

print(df.columns)

print("\nMissing Values:")

print(df.isnull().sum())

print("\nSummary Statistics:")

print(df.describe())

print("\nNumber of Samples in Each Class:")

print(df["Target"].value_counts())
```

Output:

```
First Five Records:
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
0                5.1               3.5                1.4               0.2
1                4.9               3.0                1.4               0.2
2                4.7               3.2                1.3               0.2
3                4.6               3.1                1.5               0.2
4                5.0               3.6                1.4               0.2

   Target
0       0
1       0
2       0
3       0
4       0

Number of Rows and Columns:
(150, 5)

Column Names:
Index(['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)',
       'petal width (cm)', 'Target'],
      dtype='object')

Missing Values:
sepal length (cm)    0
sepal width (cm)     0
petal length (cm)    0
petal width (cm)     0
Target               0
dtype: int64
```

```
Summary Statistics:
       sepal length (cm)  sepal width (cm)  petal length (cm)  \
count         150.000000        150.000000         150.000000
mean            5.843333          3.057333           3.758000
std             0.828066          0.435866           1.765298
min             4.300000          2.000000           1.000000
25%             5.100000          2.800000           1.600000
50%             5.800000          3.000000           4.350000
75%             6.400000          3.300000           5.100000
max             7.900000          4.400000           6.900000

       petal width (cm)      Target
count        150.000000  150.000000
mean           1.199333    1.000000
std            0.762238    0.819232
min            0.100000    0.000000
25%            0.300000    0.000000
50%            1.300000    1.000000
75%            1.800000    2.000000
max            2.500000    2.000000

Number of Samples in Each Class:
Target
0    50
1    50
2    50
Name: count, dtype: int64
```